Machine Learning

Deep learning: Regularization for neural networks

Sebastian Engelke

Master in Business Analytics

UNIVERSITÉ DE GENÈVE

Parameter initialization

- Fitting of deep neural networks is a `difficult task`, and the underlying theory is not well-understood.
- Therefore, most rules and recommendations are `heuristic` and it requires a good amount of `engineering` to obtain good solutions.

Parameter initialization

- Fitting of deep neural networks is a difficult task, and the underlying theory is not well-understood.
- Therefore, most rules and recommendations are heuristic and it requires a good amount of engineering to obtain good solutions.
- It is usually advisable to standardize the inputs to have mean zero and standard deviation one, so that all inputs are treated equally, especially in regularization.

Parameter initialization

- Fitting of deep neural networks is a difficult task, and the underlying theory is not well-understood.
- Therefore, most rules and recommendations are heuristic and it requires a good amount of engineering to obtain good solutions.
- It is usually advisable to standardize the inputs to have mean zero and standard deviation one, so that all inputs are treated equally, especially in regularization.

Initial parameters

- Since the function $J(\theta)$ is non-convex, the starting values $\theta^{(0)}$ can have a strong influence on the final solution.

Parameter initialization

- Fitting of deep neural networks is a difficult task, and the underlying theory is not well-understood.
- Therefore, most rules and recommendations are heuristic and it requires a good amount of engineering to obtain good solutions.
- It is usually advisable to standardize the inputs to have mean zero and standard deviation one, so that all inputs are treated equally, especially in regularization.

Initial parameters

- Since the function $J(\theta)$ is non-convex, the starting values $\theta^{(0)}$ can have a strong influence on the final solution.
- The weights and biases are usually initialized as small random values of a normal or uniform distribution.

Parameter initialization

- Fitting of deep neural networks is a difficult task, and the underlying theory is not well-understood.
- Therefore, most rules and recommendations are heuristic and it requires a good amount of engineering to obtain good solutions.
- It is usually advisable to standardize the inputs to have mean zero and standard deviation one, so that all inputs are treated equally, especially in regularization.

Initial parameters

- Since the function $J(\theta)$ is non-convex, the starting values $\theta^{(0)}$ can have a strong influence on the final solution.
- The weights and biases are usually initialized as small random values of a normal or uniform distribution.
- Larger starting weights help to avoid losing signal during forward and back-propagation.
- Too large weights, however, result in exploding values during forward propagation.

Regularization

- Typically, a deep neural network has too many parameters and will overfit the data at the global minimum of $J(\theta)$.
- In order to make the network more robust and to improve the generalization performance there are several possibilities to do regularization.

Regularization

- Typically, a deep neural network has too many parameters and will overfit the data at the global minimum of $J(\theta)$.
- In order to make the network more robust and to improve the generalization performance there are several possibilities to do regularization.

Parameter penalties

- Similarly to lasso and ridge regression, we introduce an extra term in the objective function that penalizes the size of the parameters:

$$J_{\text{reg}}(\theta) = J(\theta) + \alpha\Omega(\theta),$$

  where $\alpha > 0$ is a tuning parameter and $\Omega(\theta)$ is a norm penalty term, e.g.,

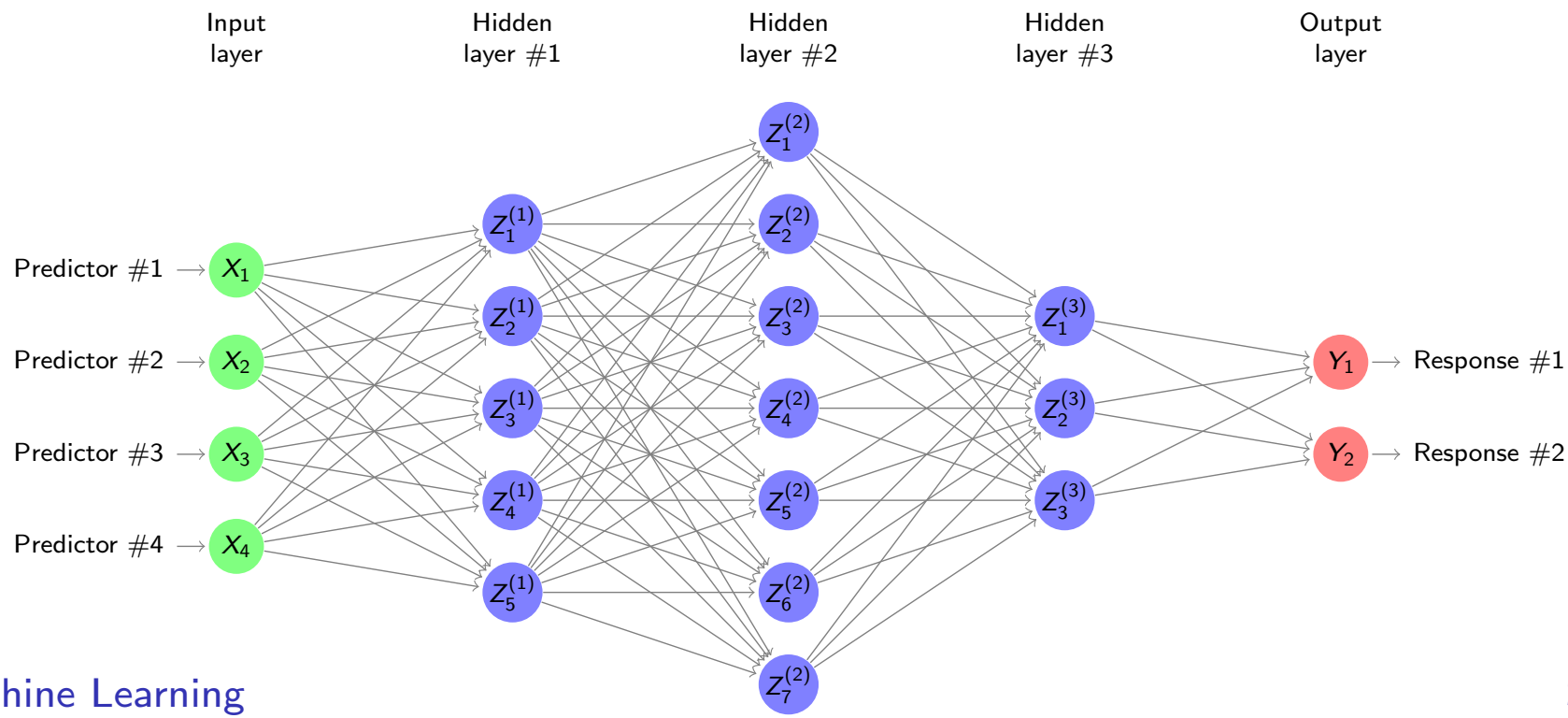
$$\Omega(\theta) = \sum_{i,j,m} {w_{ij}^{(m)}}^2,$$

  or some other norm such as ℓ_1.
- The interpretation is similar as for regularized linear models, and the tuning parameter is also chosen by cross-validation.

Regularization

Dropout

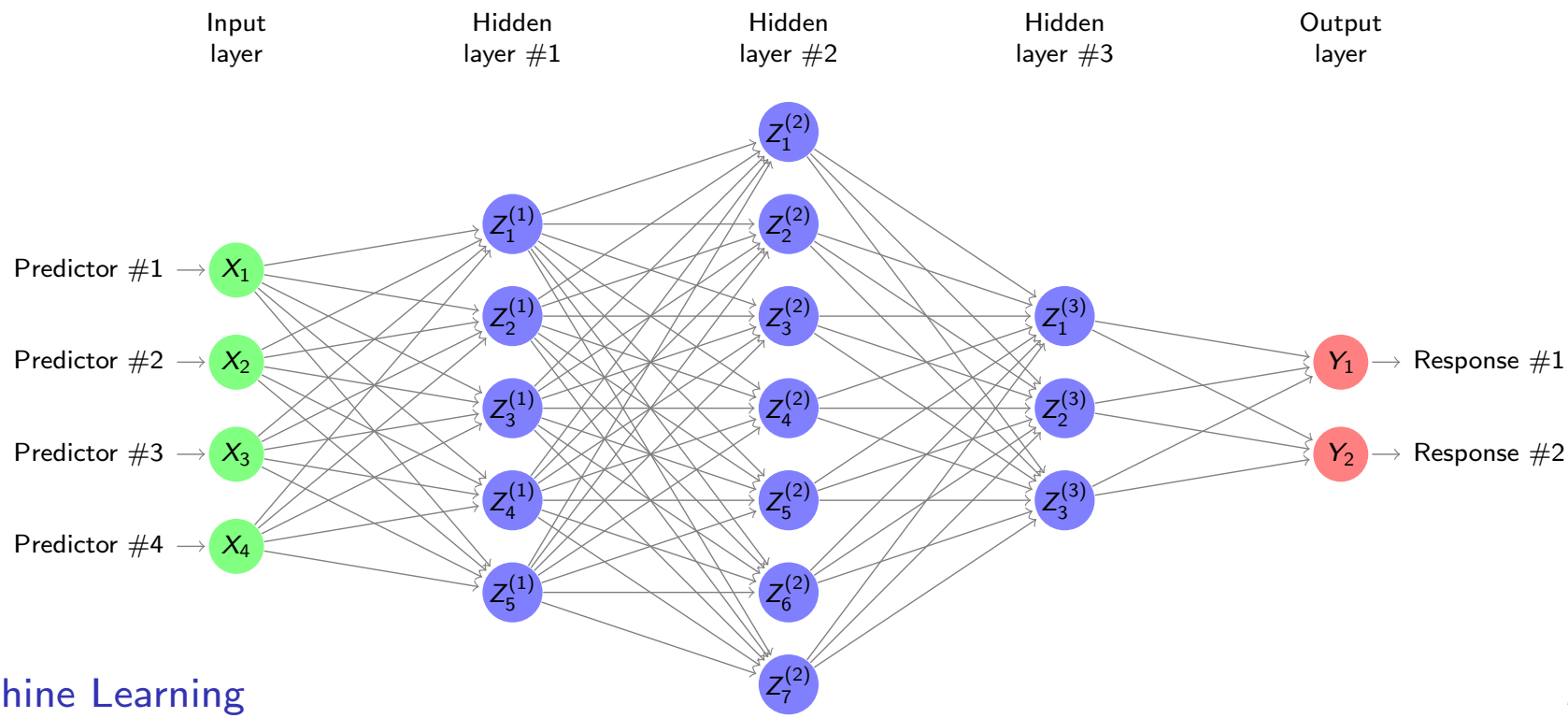
- `Dropout` is a regularization method where in each update step in the gradient descent, the `network structure` is changed.



Regularization

Dropout

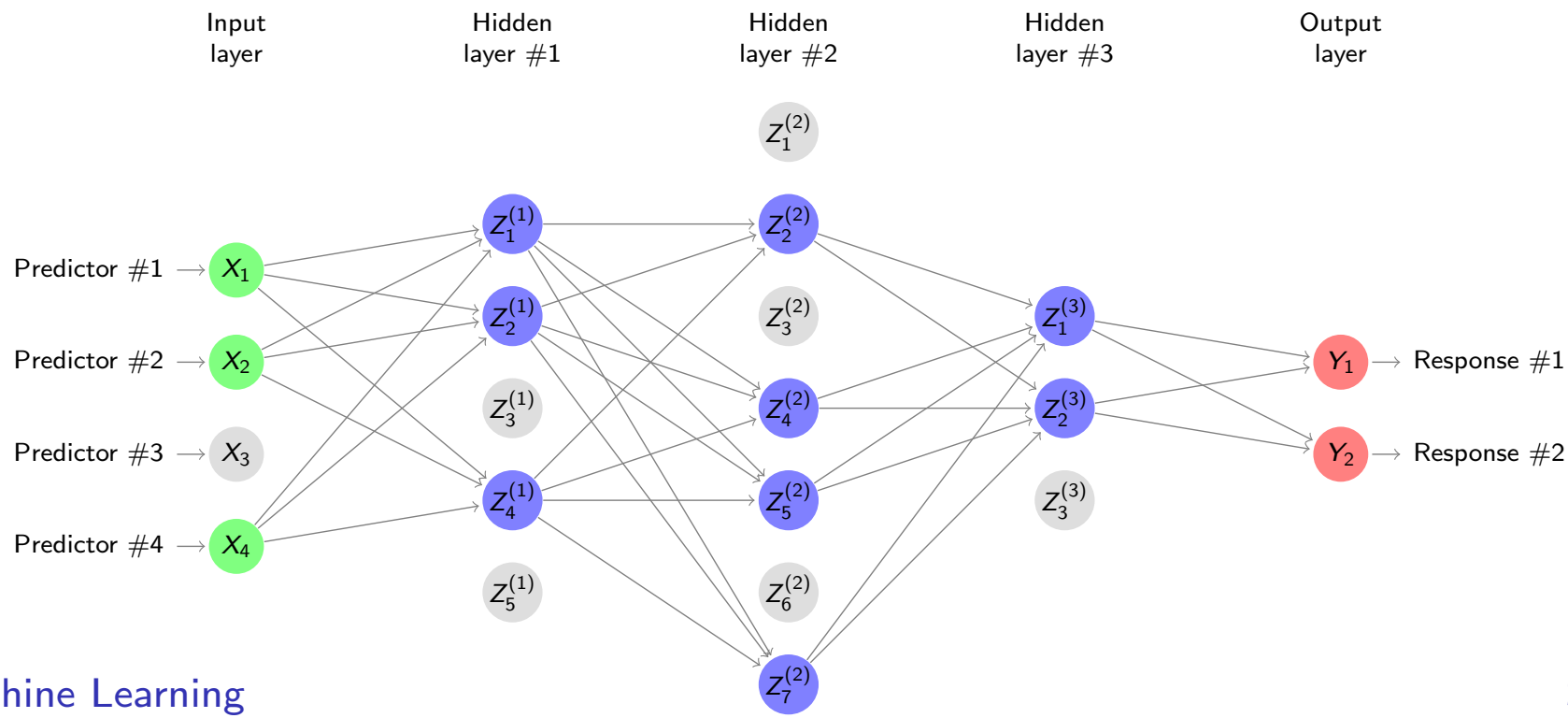
- `Dropout` is a regularization method where in each update step in the gradient descent, the `network structure` is changed.
- In fact, before a forward pass and back-propagation, each `non-output neuron` is `independently` dropped out with some probability $q \in [0, 1]$.
- For `hidden units` this probability is usually $q = 0.5$, and for `input units` $q = 0.2$.



Regularization

Dropout

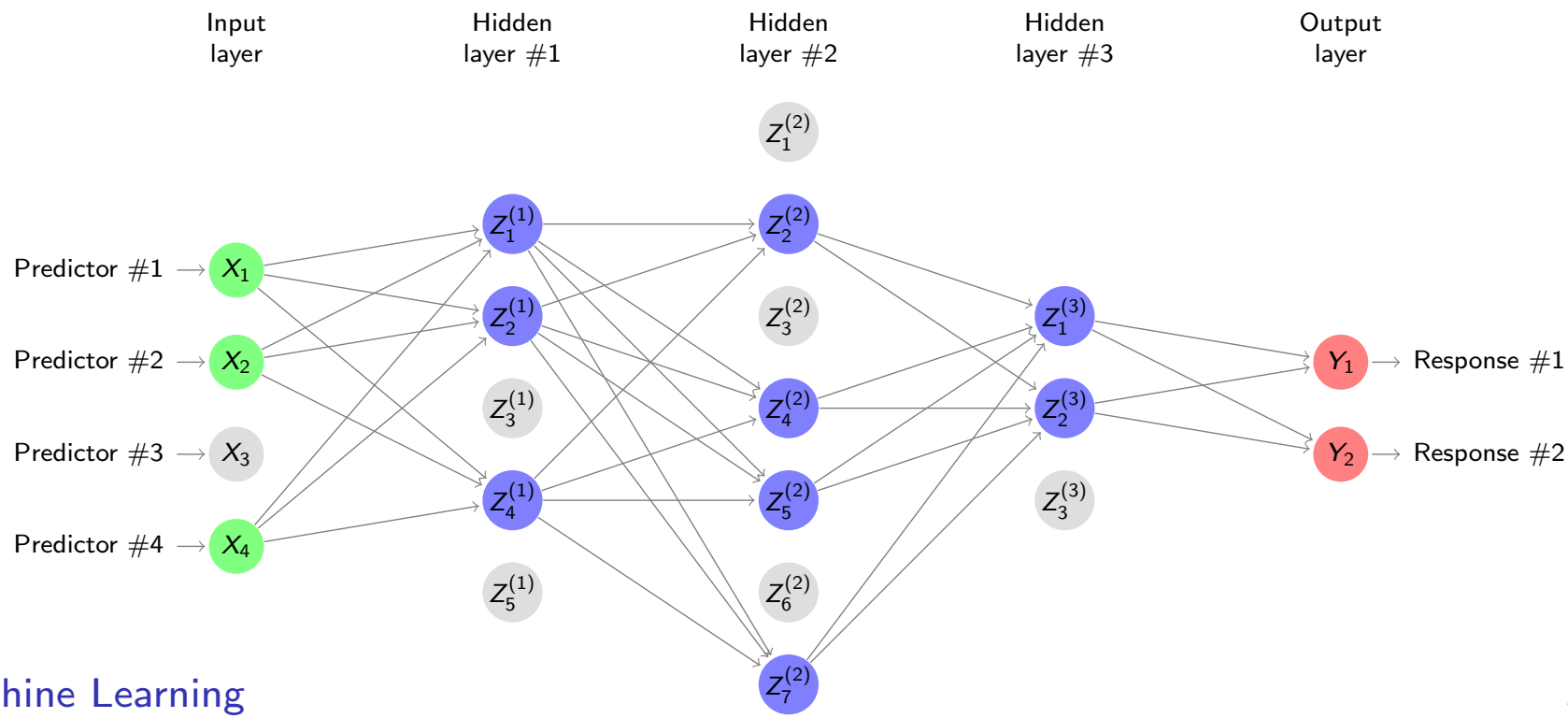
- `Dropout` is a regularization method where in each update step in the gradient descent, the `network structure` is changed.
- In fact, before a forward pass and back-propagation, each `non-output neuron` is `independently` dropped out with some probability $q \in [0, 1]$.
- For `hidden units` this probability is usually $q = 0.5$, and for `input units` $q = 0.2$.



Regularization

Dropout

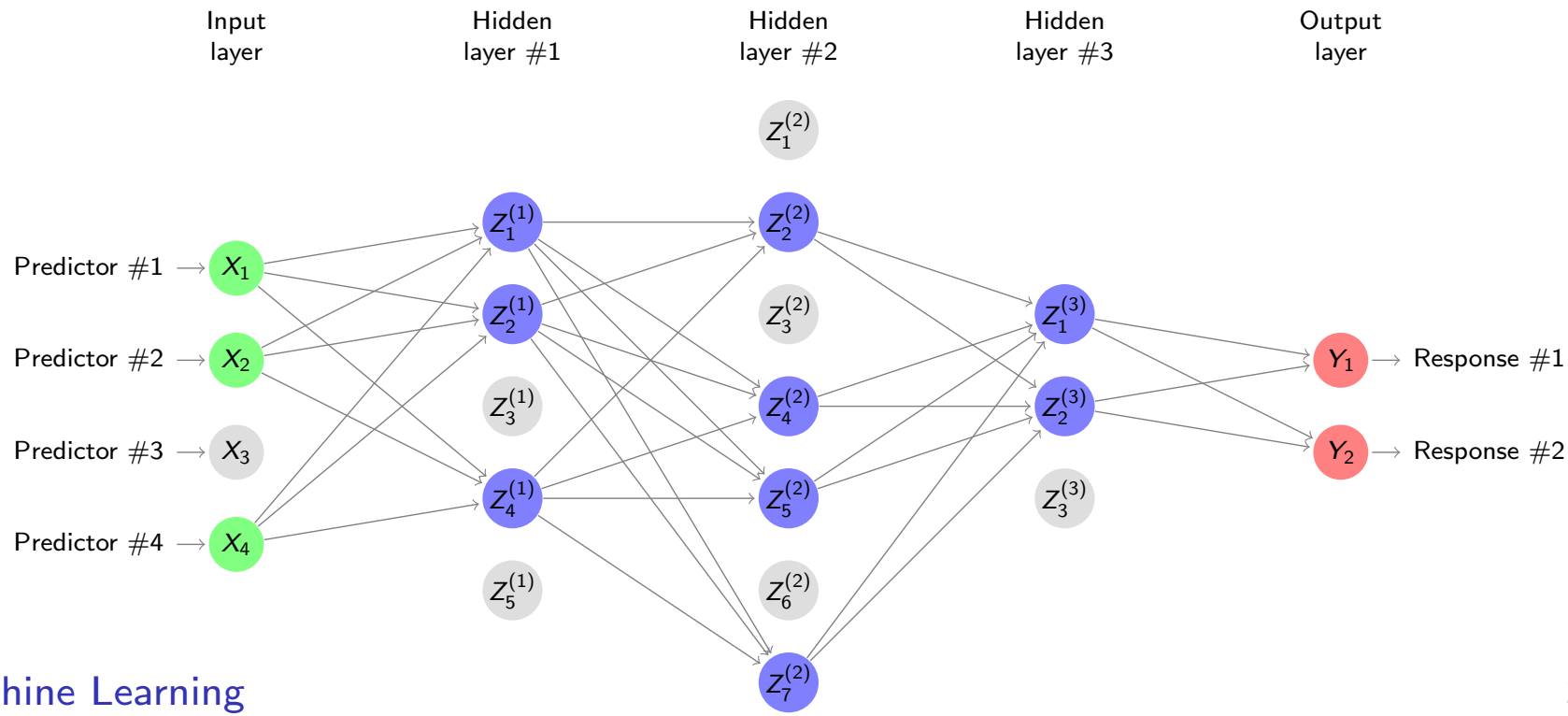
- `Dropout` is a regularization method where in each update step in the gradient descent, the `network structure` is changed.
- In fact, before a forward pass and back-propagation, each `non-output neuron` is `independently` dropped out with some probability $q \in [0, 1]$.
- For `hidden units` this probability is usually $q = 0.5$, and for `input units` $q = 0.2$.
- This `robustifies` the fitting, since every unit must learn to work independently from the dropped units.



Regularization

Dropout

- Dropout is a regularization method where in each update step in the gradient descent, the network structure is changed.
- In fact, before a forward pass and back-propagation, each non-output neuron is independently dropped out with some probability $q \in [0, 1]$.
- For hidden units this probability is usually $q = 0.5$, and for input units $q = 0.2$.
- This robustifies the fitting, since every unit must learn to work independently from the dropped units.
- Dropout is a type of bagging, where the final model is a combination of the fitted weights and the dropout probabilities.



Regularization

Early stopping

- Probably the most commonly used method for regularization is `early stopping` in the gradient descent algorithm.
- For large networks with enough `capacity to overfit` the training error decreases steadily over time, but the `validation error` begins to raise at some point.

Early stopping

- Probably the most commonly used method for regularization is `early stopping` in the gradient descent algorithm.
- For large networks with enough `capacity to overfit` the training error decreases steadily over time, but the `validation error` begins to raise at some point.
- Early stopping means to return the parameter value $\theta^{(i^*)}$ at step i^* where a `minimum` of the `validation error` is reached, and then `stop the descent`.
- This means we do not go all the way to the (local) minimum of $J(\theta)$, but stop at a parameter set that gives the `best validation error`.

Early stopping

- Probably the most commonly used method for regularization is `early stopping` in the gradient descent algorithm.
- For large networks with enough `capacity to overfit` the training error decreases steadily over time, but the `validation error` begins to raise at some point.
- Early stopping means to return the parameter value $\theta^{(i^*)}$ at step i^* where a `minimum` of the `validation error` is reached, and then `stop the descent`.
- This means we do not go all the way to the (local) minimum of $J(\theta)$, but stop at a parameter set that gives the `best validation error`.
- In some cases, early stopping can be shown to be `equivalent` to ℓ_2 regularization.

Regularization

Early stopping

- Probably the most commonly used method for regularization is early stopping in the gradient descent algorithm.
- For large networks with enough capacity to overfit the training error decreases steadily over time, but the validation error begins to raise at some point.
- Early stopping means to return the parameter value $\theta^{(i^*)}$ at step i^* where a minimum of the validation error is reached, and then stop the descent.
- This means we do not go all the way to the (local) minimum of $J(\theta)$, but stop at a parameter set that gives the best validation error.
- In some cases, early stopping can be shown to be equivalent to ℓ_2 regularization.

Parameter sharing

- Due to invariances or other domain knowledge, often it makes sense to force sets of parameters to have the same value.
- This can drastically reduce the number of unique parameters and thus performs regularization.

Regularization

Early stopping

- Probably the most commonly used method for regularization is early stopping in the gradient descent algorithm.
- For large networks with enough capacity to overfit the training error decreases steadily over time, but the validation error begins to raise at some point.
- Early stopping means to return the parameter value $\theta^{(i^*)}$ at step i^* where a minimum of the validation error is reached, and then stop the descent.
- This means we do not go all the way to the (local) minimum of $J(\theta)$, but stop at a parameter set that gives the best validation error.
- In some cases, early stopping can be shown to be equivalent to ℓ_2 regularization.

Parameter sharing

- Due to invariances or other domain knowledge, often it makes sense to force sets of parameters to have the same value.
- This can drastically reduce the number of unique parameters and thus performs regularization.
- The most popular use of this parameter sharing occurs in convolutional neural networks applied to image recognition.

Regularization

Early stopping

- Probably the most commonly used method for regularization is early stopping in the gradient descent algorithm.
- For large networks with enough capacity to overfit the training error decreases steadily over time, but the validation error begins to raise at some point.
- Early stopping means to return the parameter value $\theta^{(i^*)}$ at step i^* where a minimum of the validation error is reached, and then stop the descent.
- This means we do not go all the way to the (local) minimum of $J(\theta)$, but stop at a parameter set that gives the best validation error.
- In some cases, early stopping can be shown to be equivalent to ℓ_2 regularization.

Parameter sharing

- Due to invariances or other domain knowledge, often it makes sense to force sets of parameters to have the same value.
- This can drastically reduce the number of unique parameters and thus performs regularization.
- The most popular use of this parameter sharing occurs in convolutional neural networks applied to image recognition.

Regularization for deep neural networks is an active domain of research; the whole Chapter 7 in the book *Deep Learning* by Goodfellow et al. (2016) is devoted to this topic.